Guide des tests QA et de la documentation Dev Hub IziLife

Architecture documentaire multi projet et procédures de recette

Ce document décrit l'organisation documentaire recommandée pour un Dev Hub multi-projet et explique comment lancer les tests techniques et Playwright d'IziLife.

Décision d'organisation

Les dépôts applicatifs contiennent le code et les tests directement couplés au code. Ils ne doivent pas recevoir les documents de pilotage, les briefs, les handoffs ou les captures de référence.

Le dossier `C:\xampp\htdocs\izilife\docs` est absent et doit le rester.

Les contenus sont séparés ainsi :

- Code applicatif et contrats techniques : dépôts FO, BO et Partner Space.
- Contexte produit : `C:\Users\alcamara\Documents\agentic_Workspace\izilife\context`.
- Scénarios Playwright : `C:\Users\alcamara\Documents\agentic_Workspace\qa-izilife`.
- Références UI durables : Dev Hub.
- Résultats temporaires d'exécution : `qa-izilife\artifacts`.

Modèle générique du Dev Hub

Le **Dev Hub** est l'application générique. Il ne doit pas être conçu autour d'IziLife. Il doit pouvoir accueillir plusieurs espaces totalement indépendants qui ne collaborent pas et ne partagent aucune donnée.

Un **espace** représente un produit, une organisation ou un monde de travail cohérent. L'espace IziLife contient plusieurs projets qui collaborent. Un autre espace peut contenir `app1`, `app2` et `app4` sans aucun lien avec IziLife.

Un **projet** représente une application ou un dépôt exploitable séparément. Dans l'espace IziLife, les projets actuels sont notamment :

- `izilife` pour le Front Office ;
- `izilife-admin` pour le Back Office ;
- `izilife-partner-admin` pour le Partner Space ;
- plus tard `izilife-kiosk` et d'autres applications ;
- le Dev Hub IziLife lui-même s'il devient un projet propre à cet espace.

Les développeurs, documents de contexte, campagnes QA et bibliothèques de connaissance utilisent des relations plusieurs-à-plusieurs. Un même développeur peut travailler sur plusieurs projets. Un même document de contexte peut être lié à plusieurs projets sans être copié.

Hiérarchie recommandée

```
Dev Hub générique
  spaces
    izilife
      space.yaml
      shared-contexts
      shared-knowledge
      projects
        izilife
        izilife-admin
        izilife-partner-admin
        izilife-kiosk
        izilife-dev-hub
    autre-univers
      space.yaml
      projects
        app1
        app2
        app4
```

La structure documentaire correspondante peut être :

```
C:\xampp\htdocs\dev-hub\docs\
  spaces\
    izilife\
      space.yaml
      knowledge\
        ui\
          REFERENCE_GALLERY.md
          assets\fo\
          assets\bo\
          assets\ps\
      projects\
        izilife\project.yaml
        izilife-admin\project.yaml
        izilife-partner-admin\project.yaml
    autre-univers\
      space.yaml
      projects\
        app1\project.yaml
        app2\project.yaml
```

Le Dev Hub doit indexer les ressources externes par chemin ou URL. Il ne doit pas recopier tous les contextes dans son propre dépôt.

Entités à prévoir dans l’application

- `Space` : univers de collaboration isolé, par exemple IziLife.
- `Project` : application ou dépôt appartenant à un espace.
- `Repository` : dépôt technique rattaché à un projet.
- `ContextSource` : fichier, dossier, URL ou ressource documentaire.
- `ProjectContextLink` : lien plusieurs-à-plusieurs entre projets et contextes.
- `KnowledgeLibrary` : bibliothèque partagée au niveau espace ou propre à un projet.
- `Developer` et `ProjectMembership` : rattachement de développeurs à plusieurs projets.
- `QaSuite` : pack de tests partagé ou propre à un projet.
- `Environment` : URLs et métadonnées de local, staging et production sans secrets.

Manifeste d’espace

```
id: izilife
name: IziLife

shared_context_sources:
  - C:\Users\alcamara\Documents\agentic_Workspace\izilife\context

shared_qa:
  playwright: C:\Users\alcamara\Documents\agentic_Workspace\qa-izilife

projects:
  - izilife
  - izilife-admin
  - izilife-partner-admin
  - izilife-kiosk
```

Manifeste de projet

Chaque projet possède un fichier `project.yaml`. Il ne contient aucun secret. Il indique où se trouvent les sources utiles au Dev Hub et aux agents.

```
id: izilife-admin
space: izilife
name: IziLife Back Office

repositories:
  main: C:\xampp\htdocs\izilife-admin

qa:
  playwright: C:\Users\alcamara\Documents\agentic_Workspace\qa-izilife

context_links:
  - ref: izilife-shared-context
  - ref: qa-context

knowledge_links:
  - ref: izilife-shared-ui

environments:
  staging: https://www.staging.izilife.co/izilife-admin
```

Les manifestes référencent les contextes externes sans les recopier dans les dépôts applicatifs. Plusieurs projets peuvent utiliser le même `ContextSource` ou la même `KnowledgeLibrary`.

Tests QA techniques

Les contrats techniques sont directement dans chaque application :

- FO : `C:\xampp\htdocs\izilife\tests`
- BO : `C:\xampp\htdocs\izilife-admin\tests`
- Partner Space : `C:\xampp\htdocs\izilife-partner-admin\tests`

Les fichiers `*SourceContractTest.php` contrôlent notamment les routes, la sécurité, le CSRF, les requêtes SQL, les champs autorisés, les formulaires, les contrôleurs et les protections des points d’entrée Agent.

Lancer un contrat BO

```
cd C:\xampp\htdocs\izilife-admin
C:\xampp\php\php.exe tests\IntercommunalityIndexSourceContractTest.php
```

Lancer un contrat Partner Space

```
cd C:\xampp\htdocs\izilife-partner-admin
C:\xampp\php\php.exe tests\PartnerPasswordResetIdentitySourceContractTest.php
```

Un contrat autonome affiche PASS et retourne le code `0`. Une anomalie produit une ou plusieurs lignes `FAIL` puis retourne le code `1`.

Contrôler la syntaxe PHP

```
C:\xampp\php\php.exe -l app\Models\Intercommunality_model.php
```

Afficher les routes CodeIgniter

```
C:\xampp\php\php.exe spark routes
```

Certains tests historiques utilisent PHPUnit et nécessitent `vendor\bin\phpunit`. Les contrats autonomes peuvent être lancés directement avec PHP.

Tests QA Playwright

Le pack navigateur se trouve ici :

```
C:\Users\alcamara\Documents\agentic_Workspace\qa-izilife
```

Les scénarios sont sous `tests` :

- `fo-campaign.spec.js` : campagne Front Office principale.
- `fo-exhaustive.spec.js` : recette Front Office étendue.
- `bo-campaign.spec.js` : connexion et routes essentielles du Back Office.
- `bo-ps-exhaustive.spec.js` : recette étendue Back Office et Partner Space.
- `bo-targeted-regressions.spec.js` : régressions ciblées du Back Office.
- `partner-campaign.spec.js` : parcours métier Partner Space.
- `partner-auth-fixtures.spec.js` : authentification séparée des six comptes Partner.

Préparer les identifiants

Les secrets sont uniquement stockés dans :

```
C:\Users\alcamara\Documents\agentic_Workspace\qa-izilife\.env
```

Ce fichier est ignoré par Git. Le fichier `.env.example` décrit les variables attendues sans contenir de secret. Chaque Partner possède sa propre variable de mot de passe.

Commandes principales

Se placer dans le pack :

```
cd C:\Users\alcamara\Documents\agentic_Workspace\qa-izilife
```

Lister tous les scénarios sans les exécuter :

```
npm run test:list
```

Lancer les smoke tests :

```
npm run test:smoke
```

Lancer toute la campagne :

```
npx playwright test --grep "@campaign" --reporter=line
```

Lancer la recette Front Office étendue :

```
npx playwright test tests\fo-exhaustive.spec.js --reporter=line
```

Lancer la recette Back Office et Partner Space :

```
npx playwright test tests\bo-ps-exhaustive.spec.js --reporter=line
```

Lancer un scénario par son nom :

```
npx playwright test --grep "Intercommunalités" --reporter=line
```

Interpréter les statuts

- PASS : le comportement demandé a réellement été observé.

- `FAIL` : une anomalie reproductible a été rencontrée.
- `BLOCKED_MISSING_ENV` : une variable privée manque dans `.env`.
- `BLOCKED_PERMISSION` : le compte de recette ne possède pas le droit nécessaire.
- `BLOCKED` pour une fixture : la donnée métier nécessaire n'existe pas dans l'environnement testé.

Un statut `BLOCKED` n'est pas un bug applicatif. Il indique que la précondition de test n'est pas disponible.

Captures et références Dev Hub

Les captures destinées aux futurs développements sont des assets de connaissance. Elles ne doivent pas être mélangées avec les artifacts Playwright, qui sont des résultats temporaires d'exécution.

Emplacement actuel :

```
C:\xampp\htdocs\izilife-dev-hub\docs\izilife\knowledge\ui
```

Emplacement cible recommandé dans le Dev Hub générique :

```
C:\xampp\htdocs\dev-hub\docs\spaces\izilife\knowledge\ui
```

Organisation :

- `assets\fo` contient les références du Front Office.
- `assets\bo` contient les références du Back Office.
- `assets\ps` contient les références du Partner Space.
- `REFERENCE_GALLERY.md` décrit chaque capture et son usage.

Déroulement conseillé d'une recette manuelle

1. Choisir le parcours métier et vérifier que les fixtures nécessaires existent.
2. Exécuter le parcours manuellement dans FO, BO ou Partner Space.
3. Noter les URL, le rôle utilisé, les données manipulées et le résultat visible.
4. Classer le résultat en `PASS`, `FAIL` ou `BLOCKED` avec une cause factuelle.
5. Créer ou compléter le scénario Playwright correspondant.
6. Ajouter un contrat technique lorsque le comportement dépend d'une route, d'une règle de sécurité, d'un modèle ou d'une requête.
7. Conserver dans Dev Hub uniquement les captures utiles comme référence UI durable.
8. Relancer le scénario et les contrats concernés après chaque correction.

État des emplacements

- `C:\xampp\htdocs\izilife\docs` est absent et doit le rester.
- Le contexte IziLife est centralisé dans `C:\Users\alcamara\Documents\agentic_Workspace\izilife\context`.
- Le pack Playwright est centralisé dans `C:\Users\alcamara\Documents\agentic_Workspace\qa-izilife`.
- Les références UI appartiennent à la bibliothèque partagée de l'espace IziLife et doivent migrer vers `docs\spaces\izilife\knowledge\ui`.